



Cahier des charges professionnel

AutoMateLab INF3421

Application Next.js de manipulation d'automates avec interface graphique type n8n

UE INF3421 : Langage Formel & Compilation

Vision : développer un laboratoire web moderne où l'étudiant construit des automates sur un canvas interactif, chaîne les opérations comme des nœuds de workflow, obtient les graphes résultats, les traces pédagogiques et les exports prêts pour le rapport.

Étudiant : AZAB A RANGA FRANCK MIGUEL

Matricule : 23V2227

Année académique : 2025-2026

Document : Cahier des charges, UX, architecture, UML, plan Cursor

Version : 2.0 - Spécification Next.js + canvas type n8n

Date : 18 juin 2026

Table des matières

Résumé exécutif	1
1 Contexte, périmètre et exigences du TP	2
1.1 Contexte INF3421	2
1.2 Périmètre fonctionnel demandé	2
1.3 Périmètre produit proposé	3
1.4 Ce qui est hors périmètre	3
2 Stack technique Next.js complète	4
2.1 Décision finale	4
2.2 Pourquoi Next.js au lieu de Vite seul	5
2.3 Architecture de dépôt recommandée	5
2.4 Commandes d'installation	6
3 Concept d'interface type n8n	7
3.1 Principe général	7
3.2 Maquette globale de l'interface	7
3.3 Design system	7
3.4 Navigation proposée	8
4 Exigences fonctionnelles détaillées	9
4.1 Acteurs	9
4.2 Backlog priorisé	9
4.3 Règles métier	10
5 Diagrammes UML et flux	11
5.1 Diagramme de cas d'utilisation	11
5.2 Flux utilisateur général	11
5.3 Architecture applicative Next.js	12
5.4 Diagramme de classes métier	12
5.5 Séquence : exécuter un workflow d'opérations	13
6 Modèle de données et contrats JSON	14
6.1 Automate canonique JSON	14
6.2 Workflow JSON	14
6.3 Types TypeScript essentiels	14
6.4 Validation minimale	15
7 Spécification algorithmique	16
7.1 Convention commune	16
7.2 États accessibles, co-accessibles et utiles	16
7.3 Complétion AFD vers AFDC	16
7.4 AFN vers AFD	16
7.5 ε -fermeture et ε -AFN vers AFD	16
7.6 Minimisation	17
7.7 Thompson pur	17
7.8 Glushkov	17
7.9 Automate vers expression régulière	17
7.10 Systèmes d'équations et Arden	17
7.11 Opérations de clôture	18

8 Composants React/Next.js à produire	19
8.1 Layout principal	19
8.2 Canvas et graphes	19
8.3 Stores Zustand	19
9 Plan d'implémentation Cursor sans gaspiller les tokens	20
9.1 Règle de travail	20
9.2 Ordre exact de développement	20
9.3 Prompt maître Cursor	20
9.4 Prompt lot 1 : initialisation UI	21
9.5 Prompt lot 2 : types et validateurs	21
9.6 Prompt lot 3 : algorithms P0	21
9.7 Prompt lot 4 : canvas automate	21
9.8 Prompt lot 5 : workflow type n8n	22
10 Tests, qualité et démonstration	23
10.1 Jeux de tests minimaux	23
10.2 Critères non fonctionnels	23
10.3 Scénario de démonstration fort	23
11 Plan du rapport final de 10 pages	25
11.1 Structure recommandée	25
11.2 Tableau de participation	25
12 Définition de terminé	26
Conclusion	27
A Annexe : checklist de livraison	28
B Annexe : exemple de README minimal	29

Résumé exécutif

Objectif principal

Réaliser une application web **Next.js complète**, belle et professionnelle, permettant de créer, visualiser, transformer, minimiser, canoniser et exporter des automates finis. L'interface doit reprendre l'esprit d'un éditeur visuel de workflow : nœuds, connexions, canvas, panneau latéral, historique d'exécution et résultats comparables.

Le TP demande une application web conviviale pour implémenter et visualiser plusieurs opérations sur les automates : résolution de systèmes d'équations, conversion AFN vers AFD, extraction d'expression régulière, complétion AFD vers AFDC, identification des états accessibles/co-accessibles/utiles, émondage, conversions avec ϵ -transitions, Thompson, Glushkov, minimisation, canonisation et opérations de clôture. Le rendu final est une application web et un rapport de groupe de 10 pages maximum, avec rapport du chef de groupe indiquant la participation de chacun.

Décision recommandée

Stack finale recommandée : Next.js App Router + TypeScript + Tailwind CSS + shadcn/ui + @xyflow/react (React Flow) + Zustand + Zod + Vitest. Le moteur algorithmique reste en TypeScript pur dans src/core. L'interface comporte deux vues fortes : un **Automate Canvas** pour les états/transitions et un **Workflow Canvas** inspiré de n8n pour chaîner les opérations.

Pourquoi cette version sera plus forte que les autres

- Elle ne se limite pas à des formulaires : elle propose un vrai canvas de graphes avec nœuds, arcs, zoom, mini-map, drag/drop et comparaison avant/après.
- Elle aide à expliquer les algorithmes : chaque opération donne une trace pédagogique exportable.
- Elle exploite Next.js pour une structure professionnelle : routes, layouts, composants server/client, route handlers optionnels et déploiement simple.
- Elle permet une démonstration impressionnante : l'utilisateur dépose un automate, ajoute un nœud « AFN vers AFD », connecte vers « Minimisation », puis lance le workflow.

Livrables ciblés

- Application web responsive et déployée avec lien public.
- Canvas interactif d'automates : création, édition, import/export JSON.
- Canvas de workflow : nœuds d'opérations, connexions, exécution étape par étape.
- Moteur algorithmique testé : conversions, ϵ -fermetures, Thompson, Glushkov, minimisation, clôture, canonisation.
- Exports : JSON, PNG/SVG, trace textuelle, snippets pour le rapport.
- Rapport final de 10 pages maximum et rapport chef de groupe.

1 Contexte, périmètre et exigences du TP

1.1 Contexte INF3421

L'UE INF3421 traite des langages formels, des automates finis, des expressions régulières et de la compilation. Le TP transforme ces notions théoriques en un outil pratique. L'application doit aider à manipuler les objets du cours tout en rendant les étapes vérifiables par l'enseignant.

1.2 Périmètre fonctionnel demandé

Fonctionnalité	Ce que l'application doit faire
Systèmes d'équations	Saisir un système d'équations de langages et afficher l'expression régulière équivalente avec étapes du lemme d'Arden.
AFN vers AFD	Construire l'AFD équivalent par ensembles d'états, avec table de transition générée.
Automate vers regex	Extraire une expression régulière à partir d'un AFD ou AFN.
AFD vers AFDC	Compléter l'automate avec un état puits si des transitions manquent.
États accessibles	Lister et colorer les états atteignables depuis l'initial.
États co-accessibles	Lister et colorer les états pouvant atteindre un final.
États utiles	Calculer l'intersection accessible/co-accessible.
Émondage	Supprimer visuellement et structurellement les états inutiles.
AFD vers AFN	Convertir un AFD en structure AFN pour uniformiser les traitements.
AFN vers ε -AFN et inverse	Ajouter ou supprimer le support de transitions spontanées quand c'est pertinent.
ε -fermeture	Calculer la fermeture d'un état ou d'un ensemble d'états.
ε -AFN vers AFD	Combiner ε -fermeture et construction par sous-ensembles.
AFD vers ε -AFN	Conversion triviale pour compatibilité de pipeline.
Thompson pur	Générer un ε -AFN depuis une expression régulière.
Minimisation	Produire l'AFD minimal équivalent.
Glushkov	Construire un automate par positions depuis une expression régulière.
Canonisation	Renommer/normaliser l'automate pour obtenir une forme stable.
Clôture	Union, intersection, complémentation, concaténation, étoile, différence.

1.3 Périmètre produit proposé

Modules de l'application

1. **Automate Studio** : édition graphique d'états et transitions.
2. **Workflow Studio** : canvas type n8n pour enchaîner des opérations.
3. **Regex Studio** : expression régulière vers automate, Thompson, Glushkov.
4. **Equation Studio** : systèmes d'équations, Arden, regex finale.
5. **Closure Studio** : opérations entre deux automates.
6. **Report Center** : export des graphes, traces et paragraphes prêts pour le rapport.

1.4 Ce qui est hors périmètre

- Authentification utilisateur obligatoire : inutile pour le TP, sauf bonus.
- Base de données obligatoire : le stockage local suffit. Option bonus avec IndexedDB.
- Collaboration temps réel : trop coûteuse pour un TP.
- Backend lourd : non nécessaire pour les calculs d'automates de taille pédagogique.

2 Stack technique Next.js complète

2.1 Décision finale

Décision recommandée

Construire l'application en **Next.js App Router**. Les pages et layouts seront dans `src/app`. Les composants interactifs de canvas seront des **Client Components**. Les algorithmes seront dans `src/core`, indépendants de React et réutilisables dans les tests.

Besoin	Technologie	Rôle dans le projet
Framework	Next.js App Router	Routing, layouts, pages, déploiement propre, route handlers optionnels.
Langage	TypeScript strict	Éviter les erreurs sur les automates, transitions et résultats.
Interface Composants UI	Tailwind CSS shadcn/ui + Radix UI	Design rapide, responsive, propre. Boutons, panels, tabs, dialogs, command palette, toast.
Icônes	lucide-react	Icônes modernes pour opérations, états et exports.
Graphes/canvas	@xyflow/react	Nœuds, arcs, zoom, mini-map, drag/drop, workflow et automate canvas.
Layout auto	dagre ou elkjs	Réorganisation automatique des automates.
État global	Zustand	Automate courant, workflow courant, résultat, historique.
Validation	Zod	Schémas JSON, erreurs lisibles, import sécurisé.
Formulaires	react-hook-form	Formulaires d'états, transitions, paramètres d'algorithme.
Tests unitaires	Vitest	Vérification du moteur algorithmique.
Tests UI bonus	Playwright	Scénarios de démonstration si le temps reste.
Export image	html-to-image / SVG	Captures des graphes pour le rapport.
Stockage local	localStorage puis IndexedDB	Sauvegarde projets et historique sans backend.
Qualité	ESLint + Prettier	Code homogène, propre pour le rendu.
Déploiement	Vercel	Lien public rapide pour démonstration.

2.2 Pourquoi Next.js au lieu de Vite seul

- Meilleure architecture de pages : /lab, /workflow, /regex, /equations, /report.
- Meilleure finition produit : metadata, layout global, navigation, pages d'aide.
- Possibilité d'ajouter des route handlers plus tard, par exemple pour générer un export serveur.
- Déploiement Vercel très direct.

2.3 Architecture de dépôt recommandée

```

automatelab-inf3421/
+-- src/
| +-- app/
| | +-- layout.tsx
| | +-- page.tsx # landing / dashboard
| | +-- lab/page.tsx # automate canvas
| | +-- workflow/page.tsx # canvas type n8n
| | +-- regex/page.tsx # regex -> automates
| | +-- equations/page.tsx # Arden / systèmes
| | +-- closure/page.tsx # opérations de clôture
| | +-- report/page.tsx # exports rapport
| | `-- api/export/route.ts # bonus: export serveur
+-- components/
| | +-- layout/ # sidebar, topbar, shell
| | +-- canvas/ # base React Flow
| | +-- automata/ # states, edges, editor
| | +-- workflow/ # operation nodes type n8n
| | +-- algorithms/ # panels de trace/resultat
| | +-- forms/ # formulaires automates
| | `-- ui/ # shadcn/ui
+-- core/ # moteur pur TypeScript
| | +-- types.ts
| | +-- validators.ts
| | +-- graph-utils.ts
| | +-- accessible.ts
| | +-- trim.ts
| | +-- complete-dfa.ts
| | +-- epsilon-closure.ts
| | +-- nfa-to-dfa.ts
| | +-- enfa-to-dfa.ts
| | +-- minimize.ts
| | +-- regex-parser.ts
| | +-- thompson.ts
| | +-- glushkov.ts
| | +-- automaton-to-regex.ts
| | +-- arden-solver.ts
| | +-- closure-operations.ts
| | `-- canonize.ts
+-- store/ # Zustand stores
+-- examples/ # exemples cours/TD
+-- lib/ # export, ids, layout auto
| `-- tests/ # tests Vitest
+-- docs/ # rapport, captures, cahier
+-- public/
+-- package.json
`-- README.md

```


2.4 Commandes d'installation

```
npx create-next-app@latest automatelab-inf3421 \
  --ts --tailwind --eslint --app --src-dir --import-alias "@/*"

cd automatelab-inf3421
npm install @xyflow/react zustand zod react-hook-form lucide-react
npm install dagre html-to-image file-saver clsx tailwind-merge
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
npx shadcn@latest init
npx shadcn@latest add button card tabs dialog sheet select input textarea badge
  scroll-area separator dropdown-menu command toast
```

3 Concept d'interface type n8n

3.1 Principe général

L'application doit avoir un style de laboratoire visuel. Le principe n'est pas de copier n8n pixel par pixel, mais de reprendre les bonnes idées : canvas infini, nœuds connectables, panneau de configuration, historique d'exécution, mini-map, zoom et logs.

Deux canvas complémentaires

- **Automate Canvas** : graphe mathématique de l'automate, avec états et transitions.
- **Workflow Canvas** : pipeline d'opérations, par exemple « Importer automate -> AFN vers AFD -> Compléter -> Minimiser -> Exporter ».

3.2 Maquette globale de l'interface

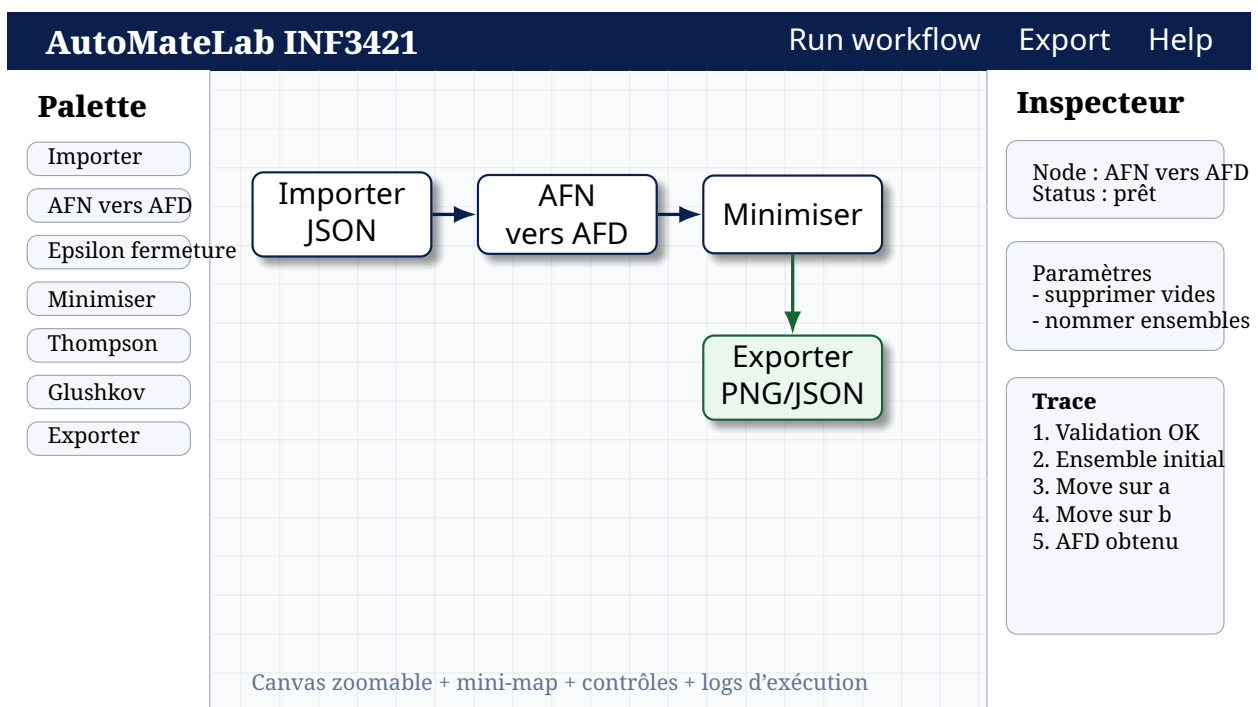


FIGURE 3.1 – Maquette d'une interface inspirée des workflows visuels de type n8n.

3.3 Design system

Élément	Spécification
Couleurs	Bleu UY1 pour navigation et actions primaires, or pour résultats importants, vert pour succès, rouge doux pour erreurs.
Canvas	Fond gris très clair quadrillé, nœuds blancs, bordures bleues, arcs courbés.

Nœud opération	Titre, icône, statut, entrée/sortie, badge de type : Automate, Regex, Trace, Export.
Nœud automate	Cercle pour état, double cercle pour final, flèche vers initial, badge ϵ si transitions spontanées.
Inspecteur	Panneau droit affichant paramètres du nœud, validation, trace et sortie.
Palette	Panneau gauche avec recherche rapide et catégories : import, conversion, analyse, regex, clôture, export.
Command palette	Raccourci Ctrl+K pour ajouter rapidement une opération.
Responsive	Sur mobile, canvas en plein écran, palettes en sheets déroulantes.

3.4 Navigation proposée

- **Dashboard** : description du projet, accès rapide aux modules, exemples de TD.
- **Automate Studio** : édition directe d'un automate.
- **Workflow Studio** : pipeline type n8n des opérations.
- **Regex Studio** : parser, Thompson, Glushkov, conversion inverse.
- **Equation Studio** : résolution de systèmes d'équations.
- **Report Center** : exports, captures, résumé des algorithmes, rapport chef de groupe.

4 Exigences fonctionnelles détaillées

4.1 Acteurs

Acteur	Rôle
Étudiant utilisateur	Créer un automate, lancer des opérations, lire les traces, exporter les résultats.
Chef de groupe	Vérifier les tâches, collecter les captures, rédiger le rapport final et le rapport de participation.
Enseignant/évaluateur	Tester la conformité, vérifier les algorithmes, juger l'ergonomie.
Système	Valider les données, exécuter les algorithmes, générer graphes et exports.

4.2 Backlog priorisé

ID	Fonctionnalité	Critère d'acceptation	Priorité
F01	Automate Canvas	Ajouter, déplacer, renommer états ; créer transitions ; marquer initial/final.	P0
F02	Workflow Canvas	Déposer des nœuds d'opérations, connecter les sorties/entrées, exécuter le pipeline.	P0
F03	Import/export JSON	Un automate exporté puis réimporté reste identique.	P0
F04	Validation	Erreurs claires : état manquant, symbole absent, AFD non déterministe, transition invalide.	P0
F05	États accessibles/co-accessibles/utiles	Listes et coloration dans le graphe.	P0
F06	Émondage	Suppression des états inutiles avec comparaison avant/après.	P0
F07	AFD vers AFDC	Ajout état puits et transitions manquantes.	P0
F08	AFN vers AFD	Construction par sous-ensembles + trace tabulaire.	P0
F09	ε -fermeture	Fermeture d'un état et d'un ensemble, affichée en table.	P0
F10	ε -AFN vers AFD	Résultat déterministe sans transition ε .	P0
F11	Minimisation	Partition initiale, raffinements, automate quotient.	P0
F12	Thompson pur	Regex vers ε -AFN, affichage construction réursive.	P1
F13	Glushkov	Regex vers automate de positions, first/last/-follow.	P1
F14	Automate vers regex	Élimination d'états ou système d'équations.	P1

F15	Arden solver	Saisie et résolution de systèmes d'équations.	P1
F16	Clôture	Union, intersection, complémentation, concaténation, étoile, différence.	P1
F17	Canonisation	Renommage stable $q_0, q_1, q_2...$ et ordre stable des transitions.	P2
F18	Exports rapport	PNG/SVG/JSON + texte de trace prêt à coller.	P2

4.3 Règles métier

1. Un AFD a un unique état initial et au plus une destination pour chaque couple (q, a) .
2. Un AFDC est un AFD où chaque état possède une transition pour chaque symbole de l'alphabet.
3. Un AFN peut avoir plusieurs destinations pour le même couple (q, a) .
4. Un ε -AFN est un AFN autorisant les transitions spontanées notées ε .
5. Un état accessible est atteignable depuis l'initial; un état co-accessible peut atteindre un final; un état utile vérifie les deux.
6. La minimisation s'applique sur un AFD, idéalement complété et émondé.
7. La complémentation exige un AFD complet avant inversion des états finaux.
8. L'interface ne doit jamais modifier l'automate source sans créer un résultat ou une sauvegarde.

5 Diagrammes UML et flux

5.1 Diagramme de cas d'utilisation

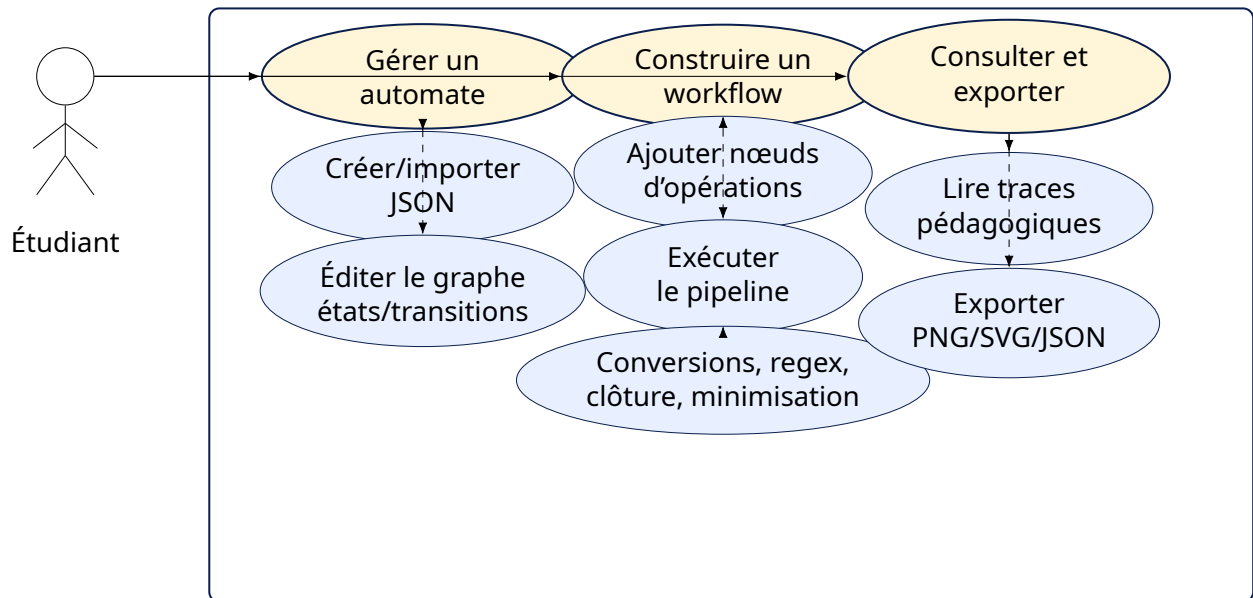


FIGURE 5.1 – Cas d'utilisation principaux.

5.2 Flux utilisateur général

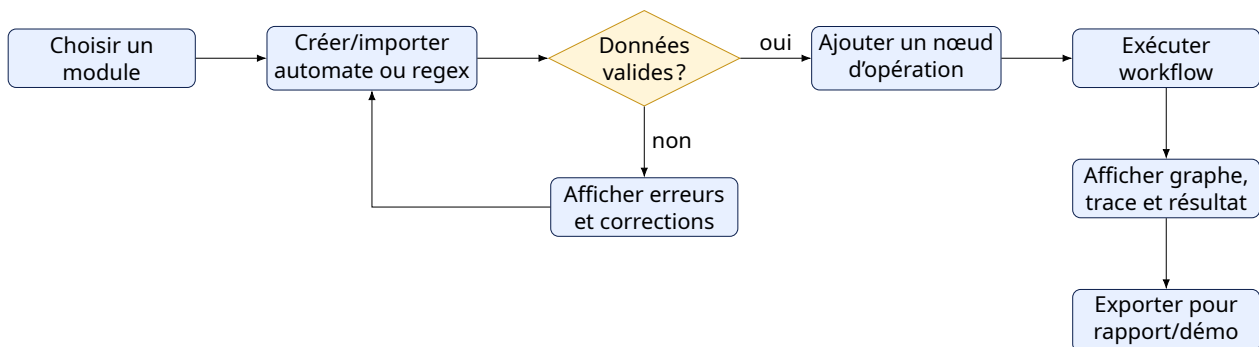


FIGURE 5.2 – Flux global de l'expérience utilisateur.

5.3 Architecture applicative Next.js

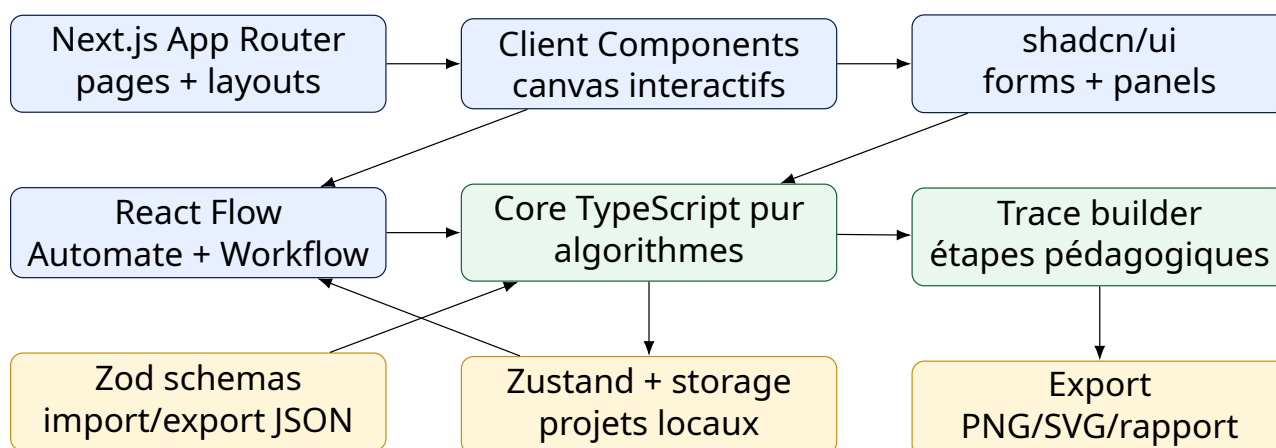


FIGURE 5.3 – Architecture modulaire Next.js.

5.4 Diagramme de classes métier

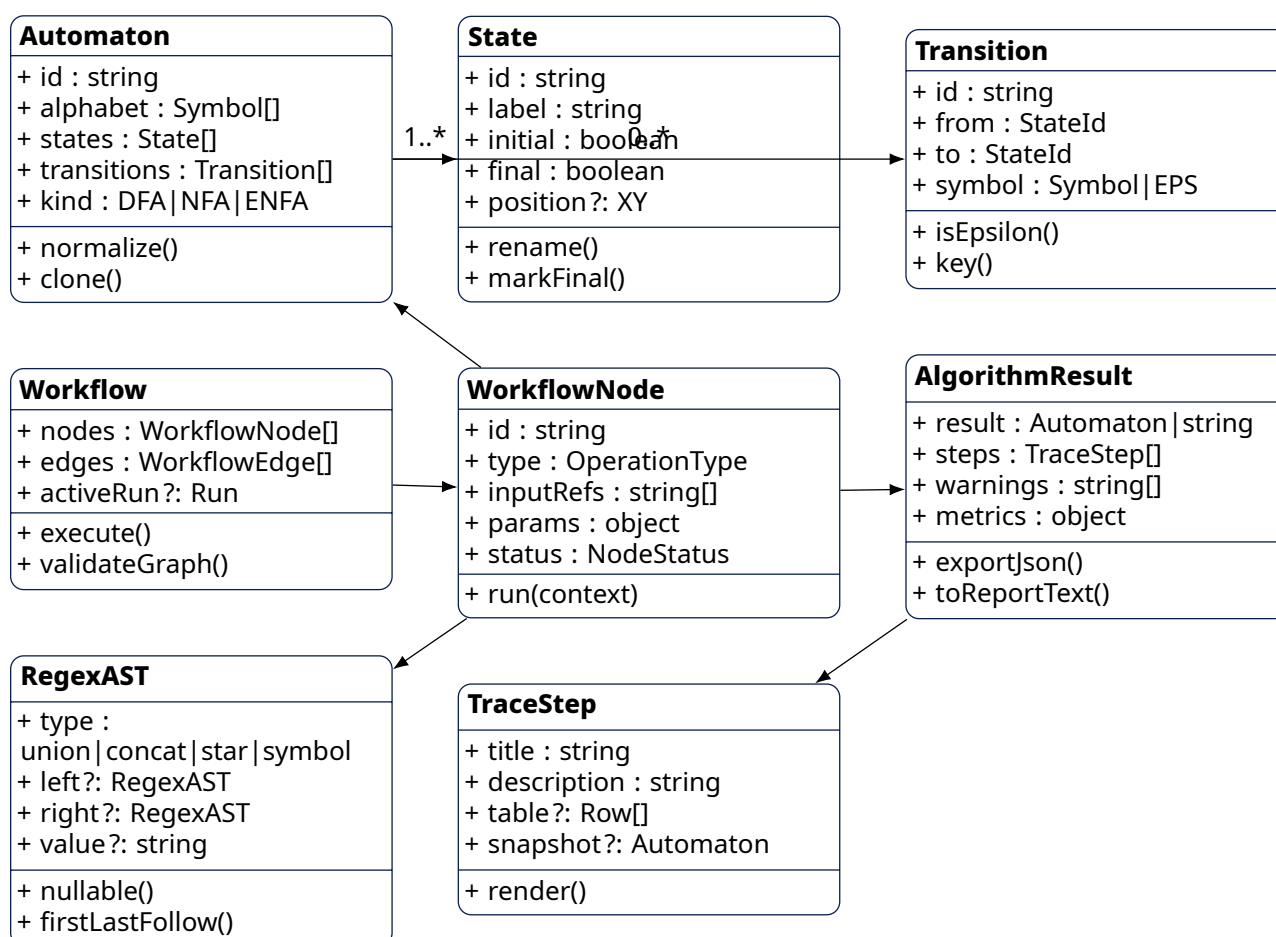


FIGURE 5.4 – Classes et types principaux du domaine.

5.5 Séquence : exécuter un workflow d'opérations

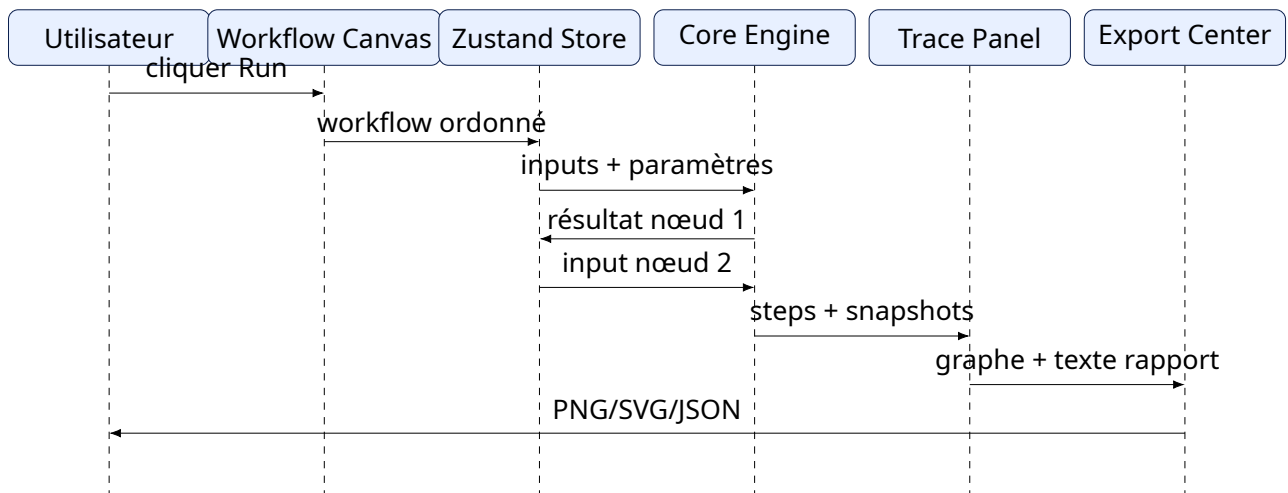


FIGURE 5.5 – Séquence d'exécution d'un workflow.

6 Modèle de données et contrats JSON

6.1 Automate canonique JSON

```
{
  "id": "A1",
  "name": "Automate exemple",
  "kind": "NFA",
  "alphabet": ["a", "b"],
  "states": [
    { "id": "q0", "label": "q0", "initial": true, "final": false, "x": 0, "y": 80 },
    { "id": "q1", "label": "q1", "initial": false, "final": false, "x": 180, "y": 80 },
    { "id": "q2", "label": "q2", "initial": false, "final": true, "x": 360, "y": 80 }
  ],
  "transitions": [
    { "id": "t1", "from": "q0", "to": "q0", "symbol": "a" },
    { "id": "t2", "from": "q0", "to": "q1", "symbol": "b" },
    { "id": "t3", "from": "q1", "to": "q2", "symbol": "b" }
  ]
}
```

6.2 Workflow JSON

```
{
  "id": "wf1",
  "name": "Conversion et minimisation",
  "nodes": [
    { "id": "n1", "type": "inputAutomaton", "position": { "x": 100, "y": 100 } },
    { "id": "n2", "type": "nfaToDfa", "position": { "x": 380, "y": 100 } },
    { "id": "n3", "type": "minimizeDfa", "position": { "x": 660, "y": 100 } },
    { "id": "n4", "type": "export", "position": { "x": 940, "y": 100 } }
  ],
  "edges": [
    { "id": "e1", "source": "n1", "target": "n2" },
    { "id": "e2", "source": "n2", "target": "n3" },
    { "id": "e3", "source": "n3", "target": "n4" }
  ]
}
```

6.3 Types TypeScript essentiels

```
export type SymbolValue = string;
export const EPSILON =  $\epsilon$  as const;
export type AutomatonKind = "DFA" | "NFA" | "ENFA";

export interface State {
  id: string;
```

```
  label: string;
  initial?: boolean;
  final?: boolean;
  x?: number;
  y?: number;
}

export interface Transition {
  id: string;
  from: string;
  to: string;
  symbol: SymbolValue | typeof EPSILON;
}

export interface Automaton {
  id: string;
  name: string;
  kind: AutomatonKind;
  alphabet: SymbolValue[];
  states: State[];
  transitions: Transition[];
}

export interface TraceStep {
  title: string;
  description: string;
  table?: Record<string, unknown>[];
  snapshot?: Automaton;
}

export interface AlgorithmResult<T = Automaton | string> {
  result: T;
  steps: TraceStep[];
  warnings: string[];
  metrics?: Record<string, number | string>;
}
```

6.4 Validation minimale

- Chaque état a un identifiant unique.
- Les transitions pointent vers des états existants.
- Les symboles de transition appartiennent à l'alphabet ou sont ϵ .
- Un AFD/AFDC ne contient pas de transition ϵ .
- Un AFD a exactement un initial et pas deux transitions conflictuelles.
- Un AFDC a toutes les transitions (q, a) définies.
- Le workflow est acyclique pour la première version afin d'éviter les boucles infinies.

7 Spécification algorithmique

7.1 Convention commune

Chaque algorithme doit être une fonction pure, sans dépendance React. Elle reçoit un automate ou une expression et retourne un `AlgorithmResult`. La trace doit permettre d'expliquer le résultat dans le rapport.

```
export function nfaToDfa(nfa: Automaton): AlgorithmResult<Automaton> {  
  // 1. validate input  
  // 2. build subset states  
  // 3. create transitions  
  // 4. return DFA + trace steps  
}
```

7.2 États accessibles, co-accessibles et utiles

1. **Accessibles** : DFS/BFS depuis l'état initial en suivant toutes les transitions.
2. **Co-accessibles** : inverser le graphe puis DFS/BFS depuis tous les états finaux.
3. **Utiles** : intersection des deux ensembles.
4. **Émondage** : conserver uniquement les états utiles et les transitions entre états utiles.

7.3 Complétion AFD vers AFDC

1. Vérifier que l'automate est déterministe.
2. Pour chaque état q et chaque symbole a , vérifier si $\delta(q, a)$ existe.
3. Si une transition manque, créer l'état puits $\perp$.
4. Ajouter les transitions manquantes vers $\perp$ et les boucles $\perp \xrightarrow{a} \perp$.

7.4 AFN vers AFD

1. L'état initial de l'AFD est $\{q_0\}$.
2. Pour un état composite S , calculer $Move(S, a)$ pour chaque symbole a .
3. Chaque ensemble non vide devient un nouvel état composite.
4. Un état composite est final si $S \cap F \neq \emptyset$.
5. La trace affiche : état composite, symbole, destination et statut final.

7.5 ε -fermeture et ε -AFN vers AFD

1. $Eclose(T)$ contient T et tous les états atteignables avec uniquement des transitions ε .
2. L'état initial de l'AFD devient $Eclose(\{q_0\})$.
3. Pour chaque composite S et symbole a , calculer $Eclose(Move(S, a))$.
4. Répéter jusqu'à stabilisation.

7.6 Minimisation

1. Compléter l'AFD si nécessaire.
2. Initialiser deux partitions : finaux et non finaux.
3. Raffiner tant que deux états d'un même bloc ne se comportent pas pareil vers les blocs cibles.
4. Construire l'automate quotient.
5. Canoniser les noms d'états pour une sortie stable.

7.7 Thompson pur

Cas	Construction
Symbole a	Deux états et une transition a .
Union $e_1 + e_2$	Nouvel initial, nouveau final, deux branches en parallèle avec transitions ε .
Concaténation $e_1 e_2$	Final de e_1 relié à initial de e_2 par transition ε .
Étoile e^*	Nouveau initial/final, acceptation de ε et boucle du final interne vers l'initial interne.

7.8 Glushkov

1. Numéroté chaque occurrence de symbole.
2. Calculer $nullab(e)$, $firstpos(e)$, $lastpos(e)$.
3. Calculer $followpos(i)$ pour chaque position.
4. Créer un état initial 0 et un état par position.
5. Créer les transitions depuis 0 vers $firstpos$, puis depuis chaque position vers $followpos$.

7.9 Automate vers expression régulière

Méthode recommandée : automate généralisé avec élimination d'états.

1. Ajouter un nouvel initial i et un nouvel final f .
2. Ajouter $i \xrightarrow{\varepsilon} q_0$ et chaque ancien final vers f .
3. Fusionner les transitions parallèles par union.
4. Pour éliminer k , mettre à jour $R_{p,r} = R_{p,r} + R_{p,k}(R_{k,k})^*R_{k,r}$.
5. L'étiquette $R_{i,f}$ est l'expression régulière finale.

7.10 Systèmes d'équations et Arden

Pour une équation de forme $X = AX + B$, si $\varepsilon \notin A$, la solution minimale est $X = A^*B$. Pour un système, on isole une variable, on applique Arden, on substitue dans les autres équations, puis on remonte.

7.11 Opérations de clôture

Opération	Méthode
Union	Nouvel initial avec transitions ε vers les deux automates ou produit selon besoin.
Intersection	Produit cartésien; final si les deux composantes sont finales.
Complément	Compléter l'AFD puis inverser finaux/non finaux.
Concaténation	Relier les finaux du premier à l'initial du second par ε .
Étoile	Ajouter nouvel initial/final et rebouclage par ε .
Différence	$L(A) \setminus L(B) = L(A) \cap \overline{L(B)}$.

8 Composants React/Next.js à produire

8.1 Layout principal

Composant	Responsabilité
AppShell	Structure globale : sidebar, topbar, zone contenu.
SidebarNav	Navigation Dashboard, Automate, Workflow, Regex, Equations, Report.
Topbar	Boutons Run, Save, Export, Theme, Help.
CommandMenu	Ajout rapide d'opérations avec Ctrl+K.
ThemeProvider	Mode clair/sombre.

8.2 Canvas et graphes

Composant	Rôle
AutomatonCanvas	Affiche les états et transitions avec React Flow.
StateNode	Nœud d'état : initial, final, utile, puits, sélectionné.
TransitionEdge	Arc courbé avec symbole, support boucles et multi-arcs.
WorkflowCanvas	Canvas type n8n pour chaîner les opérations.
OperationNode	Nœud d'opération avec entrée, sortie, statut, icône.
NodeInspector	Paramètres et résultats du nœud sélectionné.
TracePanel	Affichage des étapes de calcul.
ComparisonView	Graphe avant/après pour conversions.

8.3 Stores Zustand

```
interface AutomatonStore {
  current: Automaton | null;
  selectedStateId?: string;
  selectedTransitionId?: string;
  setAutomaton(a: Automaton): void;
  addState(s: State): void;
  addTransition(t: Transition): void;
  applyResult(result: Automaton): void;
}

interface WorkflowStore {
  workflow: Workflow;
  selectedNodeId?: string;
  runHistory: Run[];
  addOperation(type: OperationType): void;
  connect(source: string, target: string): void;
  runWorkflow(): Promise<void>;
}
```

9 Plan d'implémentation Cursor sans gaspiller les tokens

9.1 Règle de travail

Décision recommandée

Ne pas demander à Cursor de générer toute l'application d'un coup. Travailler par lots courts : types, validateurs, algorithmes, tests, puis UI. Chaque demande doit préciser les fichiers à modifier et les critères d'acceptation.

9.2 Ordre exact de développement

Lot	Objectif	Fichiers principaux	Durée
1	Initialisation Next.js	App Router, Tailwind, shadcn, layout, thème	0.5 j
2	Types + validation	src/core/types.ts, validators.ts, tests	0.5 j
3	Core graph utils	parcours, transitions, normalisation, canonisation simple	0.5 j
4	États utiles	accessible, co-accessible, useful, trim	0.5 j
5	AFDC + AFN vers AFD	complete-dfa, nfa-to-dfa, traces	1 j
6	ε -fermeture + ε -AFN	epsilon-closure, enfa-to-dfa	1 j
7	Minimisation	partitions, quotient, tests	1 j
8	Regex parser	tokens, AST, erreurs claires	1 j
9	Thompson + Glushkov	constructions et traces	1.5 j
10	Automate vers regex + Arden	élimination d'états, systèmes	1.5 j
11	Automate Canvas	React Flow, StateNode, TransitionEdge	1.5 j
12	Workflow Canvas	OperationNode, connections, run engine	1.5 j
13	Finition	exports, dashboard, responsive, README, déploiement	1 j

9.3 Prompt maître Cursor

Tu es un ingénieur Next.js/TypeScript senior. Nous développons AutoMateLab INF3421.
Stack: Next.js App Router + TypeScript strict + Tailwind + shadcn/ui + @xyflow/react + Zustand + Zod + Vitest.
Objectif: application web de manipulation d'automates avec une interface type n8n: canvas, œnuds d'opérations, connexions, traces et exports.
Contraintes:
- Le moteur algorithmique dans src/core doit rester pur, sans React.
- Chaque algorithme retourne AlgorithmResult avec result, steps, warnings, metrics.
- Ajouter des tests Vitest pour chaque fonction core.
- Les composants canvas doivent être des Client Components.

- Ne modifie que les fichiers demandés.
- Priorité: code lisible, typé, modulaire, testable.

9.4 Prompt lot 1 : initialisation UI

Crée l'architecture Next.js suivante:

- src/app/layout.tsx, page.tsx
 - src/app/lab/page.tsx
 - src/app/workflow/page.tsx
 - src/app/regex/page.tsx
 - src/app/equations/page.tsx
 - src/app/report/page.tsx
 - src/components/layout/AppShell.tsx, SidebarNav.tsx, Topbar.tsx
- Utilise Tailwind + shadcn/ui. Design bleu UY1, blanc, gris clair, or discret.
Ajoute une sidebar, une topbar avec Run, Export, Help, Theme.
Ne branche pas encore les algorithmes.

9.5 Prompt lot 2 : types et validateurs

Crée src/core/types.ts et src/core/validators.ts.

Implémente Automaton, State, Transition, Workflow, WorkflowNode, TraceStep, AlgorithmResult.

Implémente validateAutomaton, isDFA, isCompleteDFA, normalizeAutomaton.

Ajoute des tests Vitest: états dupliqués, transition vers état inconnu, symbole hors alphabet, epsilon dans AFD, AFD non déterministe.

9.6 Prompt lot 3 : algorithms P0

Implémente dans src/core:

- accessible.ts: getAccessibleStates, getCoAccessibleStates, getUsefulStates
- trim.ts: trimAutomaton
- complete-dfa.ts: completeDfa
- epsilon-closure.ts: epsilonClosure
- nfa-to-dfa.ts: nfaToDfa
- enfa-to-dfa.ts: enfaToDfa

Chaque fonction doit retourner ou produire une trace pédagogique.

Ajoute des tests pour chaque cas.

9.7 Prompt lot 4 : canvas automate

Crée un AutomatonCanvas avec @xyflow/react.

Fichiers:

- src/components/automata/AutomatonCanvas.tsx
- StateNode.tsx
- TransitionEdge.tsx
- AutomatonToolbar.tsx
- AutomatonInspector.tsx

Exigences:

- état initial: badge/arrow "start"
- état final: double cercle visuel

- état utile: vert, inutile: gris/rouge doux
- support transitions boucles et transitions multiples
- boutons: auto layout, zoom, fit view, export image

9.8 Prompt lot 5 : workflow type n8n

Crée WorkflowCanvas avec @xyflow/react.
Fichiers:

- src/components/workflow/WorkflowCanvas.tsx
- OperationNode.tsx
- OperationPalette.tsx
- NodeInspector.tsx
- RunTracePanel.tsx

Nuds: InputAutomaton, NfaToDfa, EnfaToDfa, CompleteDfa, Trim, Minimize, Thompson, Glushkov, Export.
Chaque nœud affiche statut: idle, running, success, error.
L'exécution suit l'ordre topologique des connexions.

10 Tests, qualité et démonstration

10.1 Jeux de tests minimaux

Test	Entrée	Résultat attendu
AFD complet	2 états, alphabet {a,b}, toutes transitions présentes	Aucun puits ajouté.
AFD incomplet	Transition manquante depuis q1 sur b	Ajout de $\perp$.
Accessibilité	q2 isolé	q2 non accessible.
Co-accessibilité	q3 ne mène à aucun final	q3 non co-accessible.
AFN vers AFD	q0 $\xrightarrow{a}$ q0 et q1	état composite {q0,q1}.
ε -fermeture	q0 $\xrightarrow{\varepsilon}$ q1 $\xrightarrow{\varepsilon}$ q2	fermeture(q0)={q0,q1,q2}.
Minimisation	deux états équivalents	fusion en un bloc.
Thompson	$(a + b)^*abb$	ε -AFN avec initial et final uniques.
Glushkov	ab^*	automate de positions sans ε .
Workflow	Input -> AFN vers AFD -> Minimize -> Export	chaque nœud passe success.

10.2 Critères non fonctionnels

Critère	Exigence
Performance	Automates de 50 états manipulables sans blocage visible.
Robustesse	Toute erreur est affichée dans un toast et dans l'inspecteur.
Lisibilité	Chaque résultat a un graphe, une table et une trace.
Responsive	Sur mobile, sidebar et inspecteur deviennent des sheets.
Maintenabilité	Fonctions core pures, tests unitaires, fichiers courts.
Accessibilité	Contraste lisible, boutons nommés, navigation clavier minimale.
Déploiement	Build Next.js sans erreur, lien public, README complet.

10.3 Scénario de démonstration fort

1. Ouvrir Dashboard et charger l'exemple $(a + b)^*abb$.
2. Dans Regex Studio, générer l'automate de Thompson.
3. Envoyer le résultat dans Workflow Studio.
4. Connecter : Thompson -> ε -AFN vers AFD -> Compléter -> Minimiser -> Canoniser -> Export.
5. Lancer le workflow et montrer chaque nœud qui passe en vert.
6. Ouvrir la comparaison avant/après pour l'AFD minimisé.

7. Exporter PNG + JSON + trace textuelle pour le rapport.

11 Plan du rapport final de 10 pages

11.1 Structure recommandée

Pages	Contenu
1	Page de garde + résumé du projet.
2	Objectifs du TP et choix de la stack Next.js.
3	Architecture globale et modèle de données.
4	Interface graphique type workflow et Automate Canvas.
5	Conversions AFN/ ε -AFN vers AFD et ε -fermetures.
6	États accessibles, co-accessibles, utiles, émondage et AFDC.
7	Minimisation, canonisation et opérations de clôture.
8	Expressions régulières : Thompson, Glushkov, automate vers regex.
9	Tests, captures, validation des résultats.
10	Conclusion, limites, perspectives.
Annexe	Rapport chef de groupe avec pourcentages de participation.

11.2 Tableau de participation

Membre	Tâches réalisées	Preuves	%
AZAB A RANGA FRANCK MIGUEL	Cahier des charges, architecture, moteur algorithmique, intégration UI, rapport	commits, captures	00
Membre 2	À compléter	commits	00
Membre 3	À compléter	commits	00
Membre 4	À compléter	commits	00

12 Définition de terminé

Critère de réussite

Une fonctionnalité est terminée seulement si elle est accessible dans l'interface, validée par des tests, documentée par une trace pédagogique, exportable et démontrable devant l'enseignant.

1. L'opération est disponible sous forme de bouton ou de nœud workflow.
2. Les entrées invalides sont détectées avant exécution.
3. Le résultat est affiché sous forme de graphe et de table.
4. La trace explique au moins les étapes principales.
5. Un test unitaires Vitest couvre un cas normal et un cas limite.
6. Le résultat peut être exporté ou copié pour le rapport.

Conclusion

Cette version du cahier des charges transforme le TP INF3421 en véritable produit web : une application Next.js moderne, visuelle et pédagogique. L'interface type n8n donne un avantage important pour la démonstration, car elle permet de chaîner les opérations sur les automates comme un pipeline clair et interactif. La stratégie de développement reste réaliste : moteur TypeScript pur, tests, composants React Flow, puis finition UI. En suivant les lots Cursor proposés, l'agent de codage peut produire rapidement un résultat beau, maintenable et supérieur à une simple application de formulaires.

A Annexe : checklist de livraison

- ✓ Le projet démarre avec `npm run dev`.
- ✓ Le build passe avec `npm run build`.
- ✓ Les tests passent avec `npm test` ou `npm run test`.
- ✓ Au moins un workflow complet fonctionne de bout en bout.
- ✓ Les captures exportées sont lisibles.
- ✓ Le README contient installation, fonctionnalités, stack, auteurs, lien déployé.
- ✓ Le rapport final fait 10 pages maximum.
- ✓ Le rapport chef de groupe est joint avec pourcentages.

B Annexe : exemple de README minimal

```
# AutoMateLab INF3421

Application web Next.js pour manipuler les automates finis et les expressions régulières.

## Stack
Next.js, TypeScript, Tailwind CSS, shadcn/ui, @xyflow/react, Zustand, Zod, Vitest.

## Installation
npm install
npm run dev

## Tests
npm run test

## Build
npm run build

## Fonctionnalités
- Automate Canvas
- Workflow Canvas type n8n
- AFN vers AFD
- epsilon-fermeture
- epsilon-AFN vers AFD
- AFD vers AFDC
- états utiles et émondage
- minimisation
- Thompson
- Glushkov
- automate vers expression régulière
- opérations de clôture

## Auteurs
AZAB A RANGA FRANCK MIGUEL et membres du groupe.
```